Fixity and precedence
Declaration spec: https://www.haskell.org/onlinereport/haskell2010/haskellch4.html#x10-800004.4

**Fixity**
aka **order of operations**
Fixity declaration includes:
1) **precedence**

| high | <-> | low |
|---|---|---|
| 9 | ~ | 0 |

2) **associativity**:
Only used to break **ties** in precedence

| | | | |
|---|---|---|---|
| left associative | a @ b @ c @ d | == | ((a @ b) @ c) @ d |
| | a + b - c + d | == | ((a + b) - c) + d |

e.g.

| | | | |
|---|---|---|---|
| right associative | a @ b @ c @ d | == | a @ (b @ (c @ d)) |
| | a \|\| b \|\| c \|\| d | == | a \|\| (b \|\| (c \|\| d)) |

e.g.

| | |
|---|---|
| none | operation sequence not allowed |
| | e.g. a == b == c is illegal |

e.g. Check fixity in ghci

```
-- use :info
ghci> :info (!!)
...
infix 6 +
```

**Resolution**
Basic resolution rules: https://www.haskell.org/onlinereport/haskell2010/haskellch3.html#x8-220003
Basic algorithm:
Suppose op1 op2:

```
if precedence op1 > precedence op2:            op1 ->
op2
else if precedence op1 < precedence op2:       op2 ->
op1
else
 if associativity op1 != associativity o2:     ERROR
 else
  else if associativity op1 == left:           op1 ->
op2
  else if associativity op1 == right:          op2 ->
op1
  else:                                        ERROR
i.e. no associativity defined
```

e.g. resolution

```
1 + 2 ^ 3 / 4 - 5 * sqrt 4
 1 + 2 ^ 3 / 4 - 5 * (sqrt 4)     precedence function
application (10)
 1 + 2 ^ 3 / 4 - 5 * 2
 1 + (2 ^ 3) / 4 - 5 * 2          precedence ^ (8)
```

```
1 + 8 / 4 - 5 * 2
1 + (8 / 4) - (5 * 2)            precedence * / (7)
1 + 2 - 10
(1 + 2) - 10                     left associative + -
3 - 10
-7
```

e.g. various

```
-- by precedence
a + b * c          ==     a + (b * c)             * (7) >
+ (6)
a + b ^ d * c      ==     a + ((b ^ d) * c)       ^ (8) >
* (7) > + (6)
f x + g y          ==     (f x) + (g y)           fn
application (10) > + (6)
f x $ h y          ==     (f x) (h y)
 function application (10) > $ (1)
x && y || z        ==     (x && y) || z           && (3)
> || (2)

-- by associativity
a / b / c      ==     (a / b) / c             left
associative
a + b - c      ==     a + (b - c)             right
associative
f . g . h      ==     f . (g . h)             right
associative
f $ g $ x      ==     f (g x)                 right
associative

-- errors
x == y == z                ERROR. no associative (== =
infix 4)
x == y && y == z           OK
l !! 0 . l !! 1            ERROR. same precedence 9 but
different fixity (!!) = infixl  (.) = infixr
(l !! 0) . (l !! 1)        OK
```

**Fixity of syntax**
Ref: https://www.haskell.org/onlinereport/exps.html
Fixity of **syntactical items** are defined in the language specification.
e.g. fixity of `let` is 10

| | | |
|---|---|---|
| $exp^{10}$ | -> \ $apat_1$ ... $apat_n$ -> exp | (lambda abstraction, n>=1) |
| | \| let decls in exp | (let expression) |

```
---- Syntax fixity
f x            10  function application
let ...          10
if ... then ...    10
case ...         10
```

```
do ...           10
-1             7   negation


---- Operator fixity
--
-- Basic
:       infixr 5 (list constructor)
-- Function
(->)    infixr -1
(.)     infixr 9
$       infixr 1
$!      infixr 1
!!      infixl 9
-- Monad
<$>     infixl 4
<*>     infixl 4
>>=     infixl 1
>>      infixl 1

-- Number
+       infixl 6
-       infixl 6
*       infixl 7
/       infixl 7
`div`   infixl 7
`mod`   infixl 7
`rem`   infixl 7
`quot`  infixl 7
^       infixr 8
^^      infixr 8
**      infixr 8

-- String
++      infixr 5

-- Comparison
&&      infixr 3
||      infixr 2
==      infix 4
!=      infix 4
<       infix 4
>       infix 4
<=      infix 4
>=      infix 4
```